Retrieval Augmented Generation

Multimedia Retrieval · Chapter 7

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

7 - Retrieval Augmented Generation 1
The RAG Pipeline 3
Why Retrieve at All 3
From Two Steps to a Pipeline 4
What RAG Adds 6
A Modular, Tunable System 6
Building the Pipeline in a Framework 6
Chunking Strategies 9
Fixed-Size Splitting 9
Sentence Boundaries 10
Splitting on Structure 11
Semantic Splitting 12
Hierarchical Chunking 13
Query-Aware Chunking 14
Which Strategy Is Best? 14
Result Generation 16
Choosing Among the Three 16
The Reader 16
The Generator 17
The Synthesizer 18
Citations 19
Agentic RAG 20
The Agent Loop 20
Walking the Loop: the 2025 Physics Nobel 21
Implementing the Loop 21
Guardrails 22
Key Findings 23
Evaluating RAG 24
Retrieval Still Sets the Ceiling 24
The Price of an Answer 24
Measuring Answer Quality 25
Comparing Two Configurations 26
When Not to Use RAG 27
Summary 29
How the Three Forms Compare 29
Key Takeaways 29
Key Formulas 29
Self-Check Questions 30
Further Reading 30

7 - Retrieval Augmented Generation

Every retrieval system so far has produced the same kind of output: a ranked list. Chapters 1 through 6 made that list better, more relevant, faster to compute, ranked by meaning as well as by words, but what the user receives is still a set of documents to open and read. The system finds where the answer probably is, and the user does the last step of extracting it.

For many searches that is exactly right. Someone shopping for a high-end gaming PC wants options to compare, not a single verdict, and a ranked list of products is the natural answer. Other questions, though, have one answer, and a list buries it. Ask "who won the 2025 Nobel Prize in Physics" and ten links with the names buried three paragraphs down is a poor experience. Here the user wants the answer, not the haystack that contains it, and the better experience is to read the results for them and return the answer directly.

Chat agents reach the same goal from the opposite direction. A language model already answers questions and synthesizes fluent replies, but only for what it saw during training. It cannot answer about this year's Nobel Prize, a company's current policies, or a document it has never read. To do that it must reach outside itself: call a search tool over a knowledge base, retrieve the relevant material, and extract or synthesize the answer from it. Grounding generation in retrieved documents this way is what the term Retrieval-Augmented Generation describes. Early RAG simply built a context by placing retrieved results in front of the model. Modern RAG is agentic: the model issues tool calls inside a reasoning loop and decides for itself when and what to retrieve. One consequence matters for this book. The model can now rewrite the query before calling the search tool, replacing the hand-built, hard-coded query transformations of earlier chapters with a learned one.

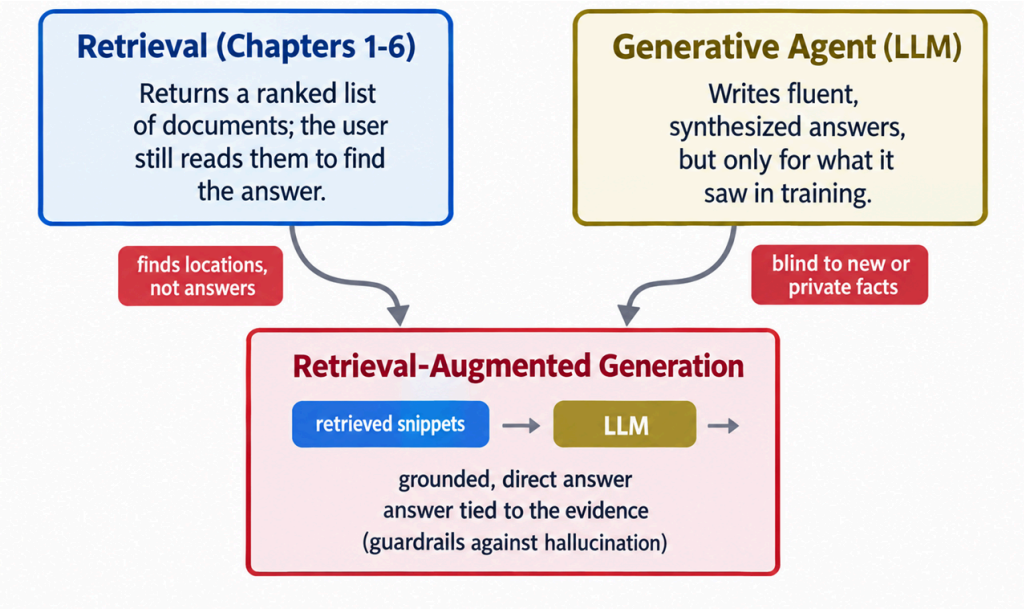


Figure 7.1: Two paths to a direct answer. Retrieval locates relevant documents but leaves the answer for the user to extract; a generative agent writes fluent answers but only for what it saw in training. RAG combines them, feeding retrieved snippets to the model to produce a grounded answer.

Put the two directions together, as in Figure 7.1, and each fixes the other's weakness. Retrieval spares the agent from reading a whole collection, by handing it only the snippets needed to form the answer. The agent spares the user from reading the result list, by turning those snippets into a direct reply. The retrieval system supplies grounded evidence, and the generator supplies the fluent, synthesized answer.

That leaves one new problem, and it is the hard one. A language model is trained to produce an answer, and it will produce a confident one whether or not it has the facts, inventing plausible but false detail. We call

this failure a hallucination. Much of what makes RAG more than "search plus a chatbot" is the machinery that keeps the generated answer tied to the retrieved evidence rather than to the model's urge to answer.

This chapter extends retrieval at the user-experience front: instead of returning only a ranked list, we want to answer the question, or surface the answer, directly. Google already does both. For a question-like query it can highlight the sentence in a top result that answers it, with no language model involved, just a direct answer lifted from the page. Its newer AI mode goes further and uses a language model to answer and elaborate. We look at how this goal reshapes retrieval, in chunking and in result generation, and how to build the whole use case end-to-end as a pipeline.

A note on units: characters, tokens, and words

This chapter mixes three ways of measuring text, and they do not line up. Humans count words. Chunking, as we do it in the next sections, often counts characters. Embedding models and the language model itself count tokens, the subword units a tokenizer produces (see Sub-Word Tokenization in Transformer Embeddings), for both the context they read and the text they generate. A token is neither a character nor a word, and the conversion is not even constant across languages. A tokenizer is trained mostly on English, so it is thriftier there: the GPT-3.5/4 tokenizer encodes the English sentence "Information retrieval finds the documents that answer a question." as 10 tokens for 9 words, about 1.1 tokens per word, while its Italian translation "Il recupero delle informazioni trova i documenti che rispondono a una domanda." takes 21 tokens for 12 words, about 1.75 tokens per word. The same meaning costs more than twice the tokens, so a fixed token budget, whether a chunk size or a context window, holds much less text in some languages than in others.

What you'll learn

- Explain why a ranked list falls short when the user wants a direct answer, and how RAG couples retrieval with a generative agent to answer and ground questions the model has not seen.
- Trace the end-to-end RAG pipeline and identify how it reuses indexing, embeddings, and reranking from earlier chapters.
- Compare chunking strategies and choose chunk sizes that serve both retrieval and generation.
- Distinguish the reader, generator, and synthesizer that turn retrieved passages into a marked span, a direct answer, or a cross-document summary.
- Explain how an agentic RAG loop takes over intent routing and query transformation, and how input and output guardrails keep it safe and grounded.
- Evaluate RAG quality with faithfulness and context metrics, and judge when RAG is not the right tool.

Prerequisites

- Index for Text Retrieval: the inverted index and BM25, which supply the sparse-retrieval arm of a RAG pipeline.
- Semantic Search: embeddings for meaning and cross-encoder reranking. See Word Embeddings and Semantic Similarity with Transformers.
- Vector Search: approximate nearest-neighbour search over embeddings, the dense-retrieval arm.
- Advanced Text Processing: tokenization and sentence segmentation, reused when splitting documents into chunks.
- Performance Evaluation: precision, recall, and nDCG, which reappear when we measure retrieval quality inside RAG.

The RAG Pipeline

The previous section made the case for answering the question instead of returning a list, by pairing a retriever with a generator. This section lays out the pipeline that does it. Most of that pipeline is machinery we have already built: the inverted index and BM25 from Index for Text Retrieval, the embeddings from Word Embeddings, and the vector search and cross-encoder reranking from Vector Search and Semantic Similarity with Transformers. Two parts are genuinely new to RAG, and each gets its own section after this one: how documents are split into chunks at two levels, and how a final stage turns retrieved passages into an answer. We start with the shape of the whole pipeline, then isolate those two additions.

A direct answer is not a new idea. Long before large language models, web search engines were already moving from lists of links toward answers a user could read without clicking through. Understanding that lineage makes the modern RAG pipeline easier to read, because each stage of it automates a step that search engines once did by hand.

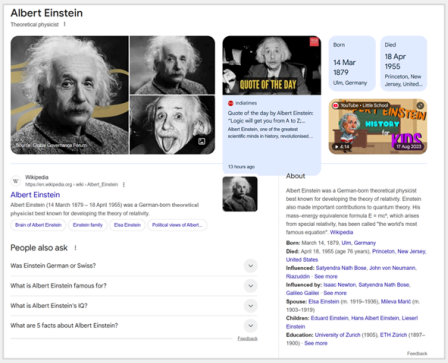
From Knowledge Graph to AI Overview: how search learned to answer (optional reading)

Google's Knowledge Graph, launched in 2012, was an early form of retrieval augmentation. Rather than returning raw documents for a query like "Albert Einstein", it supplied structured, authoritative facts: birth date, field, key contributions, related people. The answer came from a curated knowledge base, not from the ranked documents, and it shifted search from "find the page" toward "state the fact".

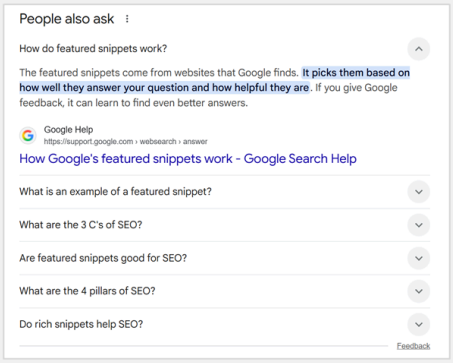
Featured Snippets followed in 2014. Here the system retrieved candidate pages the usual way, then scanned them for the passage that best answered the query and promoted that passage above the organic results. This is extractive question answering deployed at web scale: query understanding, document retrieval, passage extraction, answer selection. The answer was copied verbatim from a source, so it was grounded by construction, but the system could not rephrase or combine passages.

AI Overview, introduced in 2024, added the generative step. It retrieves relevant web content and passes it to a language model, which composes a fluent answer grounded in those sources. The three features trace the same arc this chapter follows: from structured facts, to extracted passages, to generated-but-grounded answers.

Knowledge Graph, Google (2012)



Featured Snippet, Google (2014)



Why Retrieve at All

A language model already contains an enormous amount of knowledge in its weights. The reason to retrieve is that this parametric knowledge has three hard limits, and all three are fixed by supplying external text at query time rather than by training.

The first limit is time. Weights are frozen at training, so anything that happened afterwards, like this year's Nobel laureates, is simply absent. The second is scope. No general model has read your company's internal policies, a specific insurance product's terms, or a private document collection, and these are exactly the sources that specialized questions depend on. The third is trust. A generation-only model produces fluent text whether or not it has the facts, so it can state a confident falsehood, a failure we call a hallucination.

Figure 7.2 shows the time limit in action: asked about an event past its training cutoff, a generation-only model cannot answer, while a retrieval-grounded system answers from current sources.

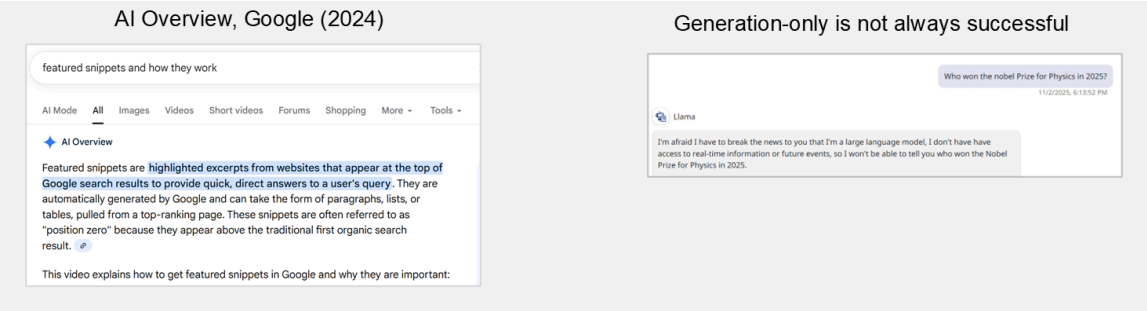


Figure 7.2: A generation-only model cannot answer a question about an event after its training cutoff (right), while a retrieval-grounded system, here Google's AI Overview, answers from retrieved web content (left).

Fine-tuning the model on the missing material is one response, but it is costly, slow, and stale the moment the underlying facts change. RAG is the cheaper and more flexible option: leave the model fixed and change what we put in front of it. Because the answer is now conditioned on retrieved passages, the model is pulled toward evidence and away from invention, and the system can cite the passages it used so a reader can check them.

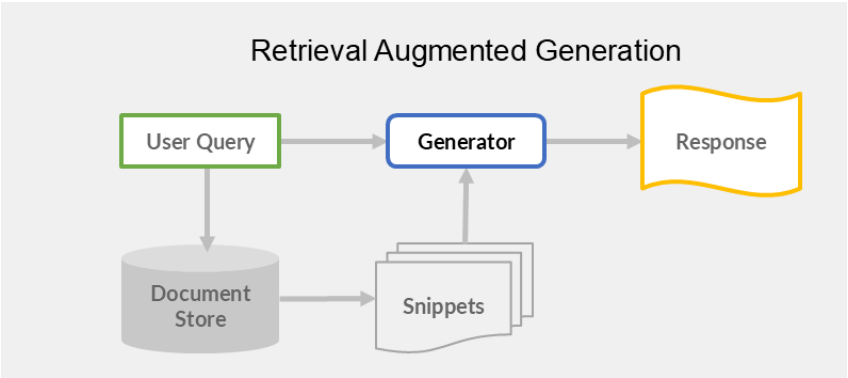


Figure 7.3: The minimal RAG loop: the query both retrieves snippets from a document store and prompts the generator, which combines query and snippets into a grounded response.

The minimal shape of RAG is small, shown in Figure 7.3. A query goes two ways at once. It prompts the generator directly, and it queries a document store, which returns the most relevant snippets. The generator then composes its answer from both the question and those snippets. Everything else in this chapter is a refinement of this loop: better ways to prepare the documents, to filter and rank what comes back, and to turn it into a trustworthy answer.

From Two Steps to a Pipeline

Early RAG was literally two steps: retrieve, then generate. It worked, but it inherited the weaknesses of each retrieval method on its own. A keyword search with BM25 (see Ranked Retrieval) matches exact terms, so a document that says "the 2025 Nobel Physics laureate was awarded to" can be missed by the query "who won the Nobel Prize", because "won" and "was awarded" share no tokens. Dense vector search (see Vector Search) matches meaning and bridges that gap, but it blurs fine detail: the word "who", which signals that the user wants a person, dissolves into the sentence embedding, so the system may return pages about the prize rather than the people who won it.

Modern systems answer this by chaining several stages, each repairing the previous one's weakness. Figure 7.4 shows the full pipeline for an informational query. We walk through it with one running example:

> Who won the Nobel Prize in Physics in 2025?

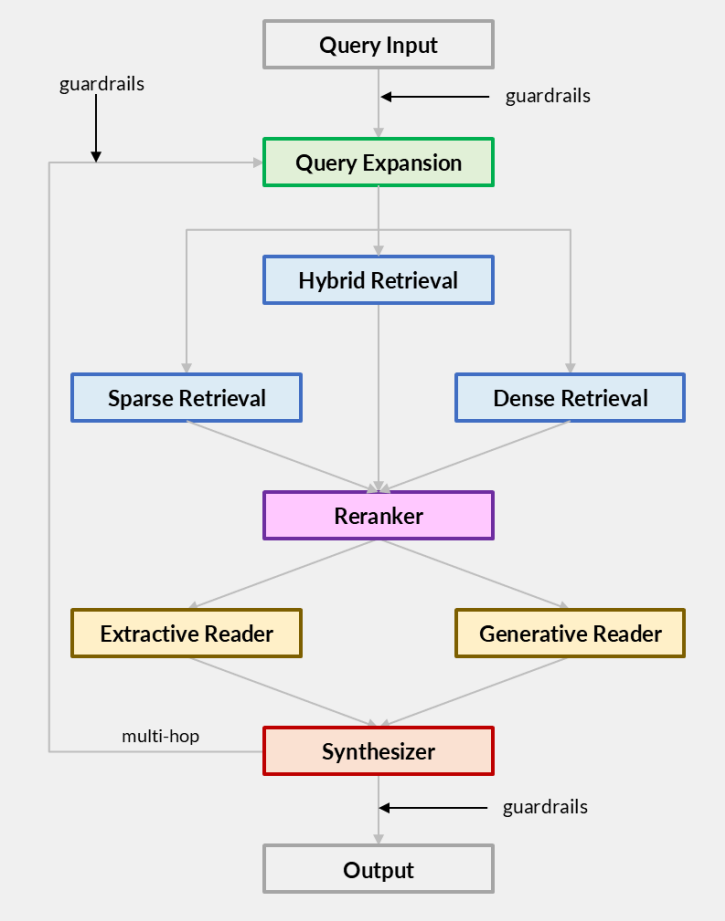


Figure 7.4: An advanced RAG pipeline: query expansion, hybrid (sparse and dense) retrieval, reranking, a reader stage, and a synthesizer, with guardrails at input and output and a multi-hop loop for iterative retrieval.

Query expansion reshapes the raw query so retrieval can succeed. It may fix spelling, add synonyms, rewrite the question, or break it into sub-questions. Our example might expand to variants like "2025 Physics Nobel winners" and "physics Nobel laureates 2025". The classical techniques for this were covered in Advanced Text Processing. Language models add rewriting patterns of their own: HyDE writes a plausible answer and searches for that instead of the question, multi-query generates several rewrites and merges their results, and step-back prompting first generalizes the query by one level. In a modern system the model itself performs the rewrite, which we return to in Agentic RAG.

Sparse and dense retrieval run the expanded query through the two complementary engines from earlier chapters. Sparse retrieval (BM25) returns matches by exact terms; dense retrieval returns matches by embedding similarity, even with different wording. Neither should return whole documents, because the generator needs a focused context of what actually matters rather than twenty pages of a PDF to wade through. Where the two engines differ is in what they demand of the split. Dense retrieval needs genuinely small chunks, since the embedding of a long passage blurs its meaning; sparse retrieval has no such need and matches terms in text of any length. RAG reconciles this with two levels: small chunks that embed cleanly and drive dense matching, and a larger parent chunk, still focused rather than a whole document, that is handed to the generator so it has enough to answer from. This two-level split is the subject of Chunking Strategies.

Hybrid retrieval merges the two result sets into one candidate pool, typically with a rank-fusion method such as reciprocal rank fusion (RRF), or by running cheap sparse retrieval first and refining its top results with the dense model. Merging is less trivial than it looks once the two engines work at different chunk levels. Rank fusion assumes both lists rank the same kind of item, so the pipeline must first bring them to a common unit, usually the parent chunk: the dense engine resolves each small chunk it matched up to the parent it belongs to, while sparse retrieval already works at that larger level. Because several small chunks often resolve to the same parent, those duplicates have to be collapsed into a single entry, keeping its best

rank or combining the scores, before fusion runs. Only then do the sparse and dense rankings line up on the same chunks and RRF can combine them. Either way the pipeline keeps a few hundred candidates where a single retriever would have kept a biased few.

Reranking applies a cross-encoder (see Semantic Similarity with Transformers) to the merged candidates. Unlike the bi-encoders used for retrieval, which embed query and document separately, a cross-encoder reads the query and a candidate together and scores their relevance directly. It is accurate enough to recognize that "who" calls for a person, and expensive enough that we apply it only to the shortlist, cutting a few hundred candidates down to a handful.

The result-generation stage turns those top passages into an answer, and it comes in three forms. A reader marks the exact span in a passage that answers the question. A generator composes a fluent, direct answer from the passages. A synthesizer summarizes across several documents, gathering evidence over multiple rounds. These three are the subject of Result Generation.

Guardrails sit at the edges of the pipeline rather than at one point in it. They constrain what enters (rejecting unsafe or out-of-scope queries) and what leaves (checking that the answer is supported by retrieved evidence before it reaches the user). The **multi-hop** loop lets the stage send a follow-up query back to the start when one retrieval pass is not enough. Both become central once an agent, rather than a fixed pipeline, drives the loop, as we see in Agentic RAG.

What RAG Adds

Reading the pipeline back, most of it is familiar. Query expansion builds on the query understanding of Advanced Text Processing; sparse, dense, hybrid retrieval and reranking are exactly the components of Chapters 4 to 6. Two stages are new, and they are new because the consumer of retrieval has changed from a human skimming a list to a model composing an answer.

The first is two-level chunking. A human reader is content with a snippet, but a generator needs enough surrounding text to compose a correct answer. RAG resolves this by splitting documents twice: small chunks that embed cleanly and drive precise retrieval, and larger parent chunks that are handed to the model to give it room to reason. The second is the result-generation stage itself: the reader, generator, and synthesizer that convert retrieved passages into a marked span, a direct answer, or a cross-document summary. Neither stage existed when the output was a ranked list. The next two sections take them in turn.

A Modular, Tunable System

The value of drawing RAG as a pipeline is that every stage is optional and swappable. A simple factual lookup can skip dense retrieval and reranking entirely and still answer correctly. A hard, domain-specific question can afford the full chain. The same architecture therefore spans a fast, cheap search tool and a slow, thorough research assistant, by turning stages on or off rather than by building a different system.

Each stage also makes a speed-for-quality trade. Sparse retrieval is fast but shallow on meaning; dense retrieval captures meaning but costs more compute; reranking raises precision but is the most expensive step per candidate. Running all three gives the best answers at the highest cost, which is why production systems route easy queries down a short path and reserve the full pipeline for queries that need it.

One consequence is worth stating plainly, because it reflects a real shift in what users will accept. Classical search chased sub-ten-millisecond latency at almost any cost. RAG relaxes that. A streamed answer that arrives over a few seconds feels natural, and for a genuinely better answer, especially in research settings, users tolerate far longer waits than a link-ranking engine could ever get away with. That tolerance is what makes a multi-stage pipeline affordable in the first place.

Building the Pipeline in a Framework

Modularity is also what makes the pipeline practical to build. Wiring retrievers, rerankers, and generators together by hand is slow and error-prone, so frameworks such as Haystack, LangChain, and LlamaIndex assemble them from reusable components. We use Haystack because its design mirrors Figure 7.4 directly: every stage is a component with typed inputs and outputs, and a pipeline is the graph that connects them.

A Haystack system has two pipelines. An indexing pipeline fills a document store: a splitter cuts documents into chunks, an embedder computes their vectors, and a writer stores both. The store can be an in-memory one for experiments or a production backend such as OpenSearch or a vector database, the systems of Index for Text Retrieval and Vector Search. The query pipeline then answers questions against that store. The sketch below builds the hybrid core of the pipeline above: BM25 and dense retrieval in parallel, fused with reciprocal rank fusion, reranked by a cross-encoder, and passed to a generator.

```
from haystack import Pipeline
from haystack.components.builders import ChatPromptBuilder
from haystack.components.joiners import DocumentJoiner
from haystack.components.retrievers.in_memory import (
    InMemoryBM25Retriever,
    InMemoryEmbeddingRetriever,
)
from haystack.dataclasses import ChatMessage
from haystack.document_stores.in_memory import InMemoryDocumentStore
from haystack_integrations.components.embedders.sentence_transformers import (
    SentenceTransformersTextEmbedder,
)
from haystack_integrations.components.rankers.sentence_transformers import (
    SentenceTransformersSimilarityRanker,
)
from haystack_integrations.components.generators.amazon_bedrock import (
    AmazonBedrockChatGenerator,
)

store = InMemoryDocumentStore()   # filled beforehand by the indexing pipeline

template = [ChatMessage.from_user("""Answer the question using only the passages
below.
{% for doc in documents %}[{{ loop.index }}] {{ doc.content }}
{% endfor %}
Question: {{ query }}""")]

rag = Pipeline()
rag.add_component("bm25", InMemoryBM25Retriever(document_store=store, top_k=50))
rag.add_component("embedder", SentenceTransformersTextEmbedder(
    model="sentence-transformers/all-MiniLM-L6-v2"))
rag.add_component("dense", InMemoryEmbeddingRetriever(document_store=store,
top_k=50))
rag.add_component("join", DocumentJoiner(join_mode="reciprocal_rank_fusion"))
rag.add_component("rerank", SentenceTransformersSimilarityRanker(
    model="cross-encoder/ms-marco-MiniLM-L-6-v2", top_k=5))
rag.add_component("prompt", ChatPromptBuilder(template=template,
                                              required_variables=["query",
"documents"]))
rag.add_component("llm", AmazonBedrockChatGenerator(model="us.anthropic.claude-
sonnet-5-5"))

rag.connect("embedder.embedding", "dense.query_embedding")
rag.connect("bm25", "join")
rag.connect("dense", "join")
rag.connect("join", "rerank")
rag.connect("rerank", "prompt.documents")
rag.connect("prompt", "llm")

question = "Who won the 2025 Nobel Prize in Physics?"
result = rag.run({
    "bm25": {"query": question},
    "embedder": {"text": question},
```

```
    "rerank": {"query": question},
    "prompt": {"query": question},
})
print(result["llm"]["replies"][0].text)
```

The Sentence-Transformers components and the model generators ship in separate integration packages (`haystack_integrations`), and the generator here is a Claude model on Amazon Bedrock. Each component maps onto a stage of the pipeline, and each can be swapped without touching the rest: a different embedding model, a stronger reranker, a production store, or another generator is a one-line change. Here both retrievers search the same chunks, so the joiner can fuse their rankings directly. With the two-level chunking of Chunking Strategies, Haystack offers a hierarchical splitter and an auto-merging retriever that match small chunks and return their parent when enough of its children are relevant, which is the reconciliation to a common unit discussed above.

The same structure is also the pipeline's limitation. The graph is fixed when it is built, and it encodes one way of answering one kind of question. A different intent needs a different graph, and a change means new code. Agentic RAG returns to this problem.

Hands-on: The RAG pipeline

Build the pipeline of this section stage by stage on a Wikipedia knowledge base about the 2025 Nobel Prizes: two-level indexing, BM25 and dense retrieval, reciprocal rank fusion, cross-encoder reranking, and a grounded answer, then wire the same pipeline in Haystack. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Chunking Strategies

Before a document can be retrieved for RAG, it has to be split into pieces. Embedding a whole book as a single vector is useless: the vector averages over so many topics that it is close to nothing in particular, and a query for one fact drowns in the rest. Splitting documents into smaller units, called chunks, is therefore the first design decision in any RAG system, and a surprisingly consequential one.

Chunking is older than RAG. Classical retrieval already split long documents so an engine could show the user the most relevant passage, with the matched terms close together, instead of a whole document to scroll through. RAG revived the technique for a harder reason: early embedding models lost accuracy beyond a few hundred tokens, and early language models had context windows too small to accept a full document. Chunking was the way to fit both.

Those technical limits have eased, but chunking has not become irrelevant. It has become a deliberate trade-off. Larger chunks give the language model more context to reason over, which helps it compose complete answers. Smaller chunks produce sharper embeddings, because a short, focused passage has one clear meaning rather than a blurred average. These pull in opposite directions, and the chunk size is where we balance them.

As The RAG Pipeline noted, the strongest resolution is not to pick one size but to keep two levels at once: small chunks that embed cleanly and drive retrieval, and larger parent chunks that expand the context handed to the generator. Most of the methods below produce one level; hierarchical chunking, later in this section, is where the two levels come together, and it is the arrangement RAG systems most often use.

The chunk-size sweet spot

In practice, chunks of roughly 512 to 1024 tokens balance the two pressures: large enough to carry a coherent idea, small enough to embed cleanly. Treat this as a starting point, not a rule. Factual lookups favour shorter chunks, because the answer sits in a small span; summarization or cross-topic questions favour longer context, because the answer must connect ideas that short fragments would separate.

Chunk sizes here are in characters, not tokens

The code below uses LangChain's `CharacterTextSplitter`, which counts characters, while the sweet spot above and a model's context window are measured in tokens. As the chapter introduction notes, a token is neither a character nor a word, and the ratio is language-specific. A rough English rule is four characters, or three-quarters of a word, per token, but set and measure chunk sizes in the tokens of the model we actually use, not in characters or words.

The sections below walk through chunking methods from the simplest to the most adaptive, using one running document throughout: the Project Gutenberg edition of *A Study in Scarlet*. Each method fixes a weakness of the one before it.

Fixed-Size Splitting

The simplest method ignores the text's structure entirely. Split the document into words, then group words until each chunk reaches a target size. To avoid cutting an idea cleanly in half at a boundary, let consecutive chunks overlap by a fixed amount, so a sentence split across the boundary still appears whole in one of the two chunks.

LangChain, one of the most widely used RAG frameworks, provides this directly:

```
from langchain_text_splitters import CharacterTextSplitter

text_splitter = CharacterTextSplitter(
    separator=" ",          # break on word boundaries
    chunk_size=1000,        # target characters per chunk
    chunk_overlap=200,      # characters shared between neighbours
```

```
    add_start_index=True,   # record each chunk's offset in the source
)
chunks = text_splitter.split_text(study_in_scarlet)
```

Applied to the running document, this produces 299 chunks that pack tightly against the limit: every one lands between 889 and 1000 characters, almost all within a few characters of 1000, as Figure 7.5 shows. The small variation comes only from the splitter stopping at the last whole word before the limit. The method is easy and fast, and it is enough when rough locality is all we need.

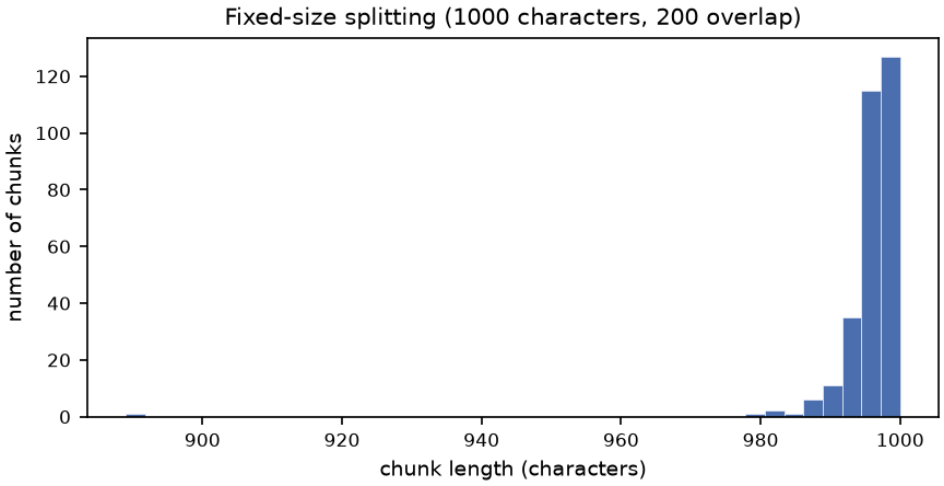


Figure 7.5: Chunk-length distribution for fixed-size splitting of A Study in Scarlet *(1000 characters, 200 overlap): nearly every chunk sits just below the limit.*

Fixed-size chunks ignore meaning

Chunk boundaries fall wherever the character count runs out, not where topics change. A new section can start halfway through a chunk and finish in the next, so a single chunk can mix the end of one idea with the start of an unrelated one. Embeddings of such chunks blend concepts, and retrieval returns passages that lack coherent context.

Sentence Boundaries

A first improvement is to treat the sentence, not the character count, as the smallest unit. We add whole sentences to a chunk until it reaches the size limit, so chunks never break mid-sentence. Chunk sizes vary a little as a result, but the pieces read more naturally and map better onto meaning.

Detecting sentence boundaries is harder than splitting on every period, because abbreviations like “Dr.” and quoted dialogue create false boundaries. The mechanics of sentence segmentation, including the Punkt and spaCy segmenters, are covered in Advanced Text Processing. Here we simply use such a segmenter as a building block:

```
from langchain_text_splitters import NLTKTextSplitter  # or SpacyTextSplitter

text_splitter = NLTKTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = text_splitter.split_text(study_in_scarlet)
```

On the running document, sentence-based splitting also yields 299 chunks, clustered just below the 1000-character limit with a mean near 930, as Figure 7.6 shows. The spread is wider than for fixed-size splitting, reaching down to about 560 characters, because a chunk closes as soon as the next whole sentence would overflow it, and sentence lengths vary. Sentence-based chunking balances structure against simplicity, but it still does not know where topics shift, so a single chunk can hold two unrelated but adjacent sentences.

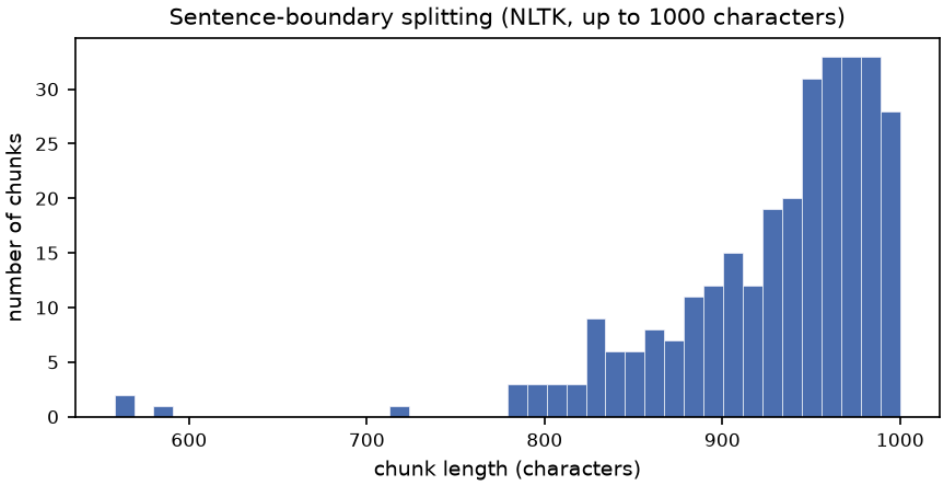


Figure 7.6: Chunk-length distribution for sentence-boundary splitting (NLTK, up to 1000 characters): a wider spread than fixed-size, since chunks close on sentence boundaries.

Splitting on Structure

Authors already mark where content changes: chapters, sections, paragraphs. Splitting on those structural elements produces chunks that align with the author's own units of thought. The signals depend on the format. In the Project Gutenberg text, more consecutive newlines mean a larger break, so four newlines mark a chapter and two mark a paragraph. In HTML or Markdown, heading tags and paragraph tags play the same role.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n\n\n", "\n\n"],  # four newlines = chapter, two = paragraph
    chunk_size=2000,      # large cap: keep each structural unit whole,
    chunk_overlap=0,      # only splitting a unit that exceeds the cap
    add_start_index=True,
)
chunks = text_splitter.split_text(study_in_scarlet)
```

Structural chunks are more coherent than word- or sentence-based splits, but their sizes are far less uniform, as the three panels of Figure 7.7 show. Splitting by chapter yields just 19 pieces (top panel), most of them enormous, between 10,000 and 23,000 characters, with a handful of tiny ones from front matter and section breaks. Such coarse chunks are too big to embed well or to sit alongside others in the model's context. Splitting by paragraph controls size better and produces 816 pieces (middle panel), but *A Study in Scarlet* is written as dialogue, so more than half the paragraphs are under 200 characters, often a single line of speech. Those tiny chunks carry too little context to retrieve or answer from.

The fix is to merge. We keep paragraph splitting but merge adjacent small chunks until each one is substantial. On the running document this lifts the typical chunk to around 850 characters and clears out the dialogue fragments (bottom panel of Figure 7.7), though a few long paragraphs still run past 1500. For overlap, a fixed character count works poorly here, because a whole paragraph can be longer than the overlap window. A better approach adds the last sentence of the previous chunk and the first sentence of the next to each chunk, carrying just enough neighbouring context across the boundary.

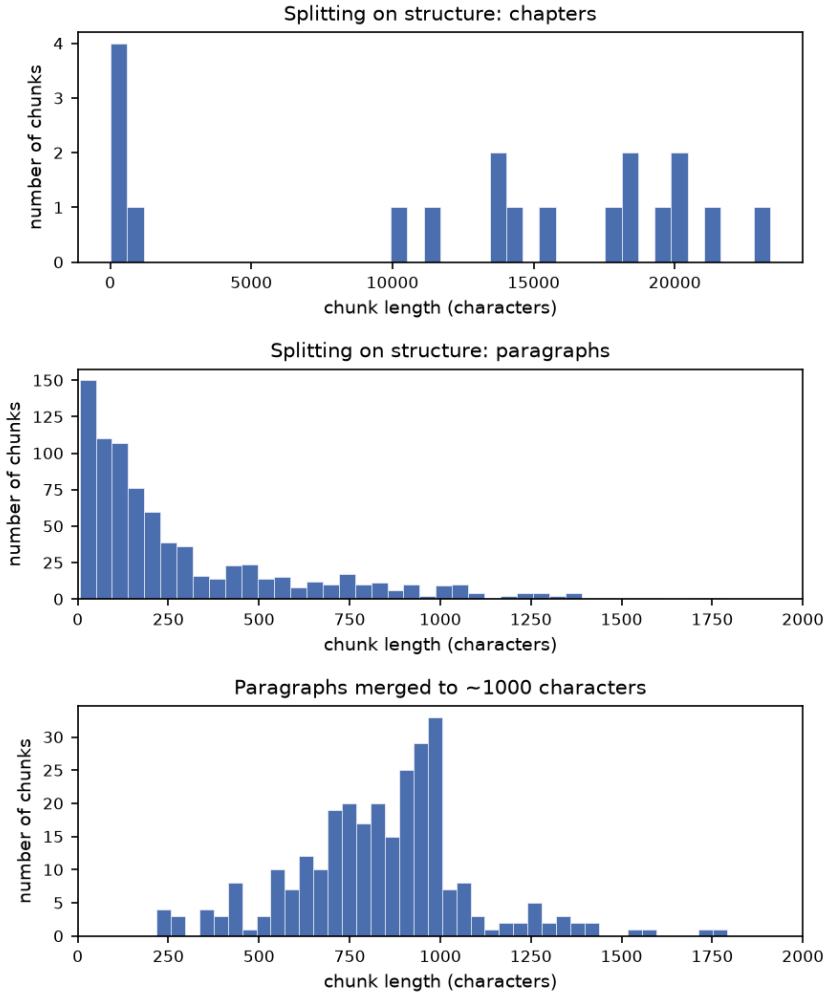


Figure 7.7: Structural splitting of A Study in Scarlet. *Chapters (top) are few and huge; paragraphs (middle) are many and mostly tiny because of dialogue; merging small paragraphs (bottom) clusters most chunks near 850 characters.*

Semantic Splitting

Structural splitting assumes reliable structure, and that assumption breaks on scraped web pages, badly laid-out PDFs, and scanned documents with no headings. Semantic splitting drops the reliance on formatting and groups sentences by meaning instead.

The method extends sentence-based splitting with a similarity test. We embed each sentence, then merge neighbouring sentences or chunks when they are similar enough and the result stays under the maximum size. Similarity must be measured in meaning, not surface words: a term-frequency vector would score two differently-worded sentences about the same idea as unrelated, so we use a bi-encoder (see Semantic Similarity with Transformers) and compare sentence embeddings with cosine similarity.

Merging proceeds iteratively. Start with each sentence as its own chunk; compute similarities between adjacent chunks; merge the most similar neighbouring pairs that stay within the size limit; repeat. Over several passes, this grows longer chunks wherever the text stays on one topic and leaves a boundary wherever the topic shifts. Semantic chunks align with meaning better than any fixed rule, at the cost of the extra computation to embed every sentence and compare neighbours.

The top panel of Figure 7.8 shows the result on the running document, and its shape is the signature of semantic splitting. The 491 chunks spread across the whole size range instead of piling up at the limit, because a boundary falls wherever the topic shifts, not where a character budget runs out, so the sizes track the content and the median lands near 540 characters. The cluster of very small chunks on the left is the method's cost: a sentence that resembles neither neighbour is left unmerged, so roughly a third of the chunks stay under 200 characters.

A simple refinement removes that cost. After the similarity merge, we make one more pass over the chunks that are still too small and fold each one into a neighbour, choosing the semantically closer of the two, and coalescing runs of adjacent fragments (the back-and-forth of dialogue) with each other first. This pass relaxes the similarity threshold, since the goal is now to eliminate fragments rather than to find strong matches, but it keeps a size ceiling. On the running document it cuts the 491 chunks to 355, removes every chunk under 200 characters, and lifts the median to about 670, as the bottom panel shows. A few chunks run slightly past 1000, where a fragment was attached to an already-full neighbour.

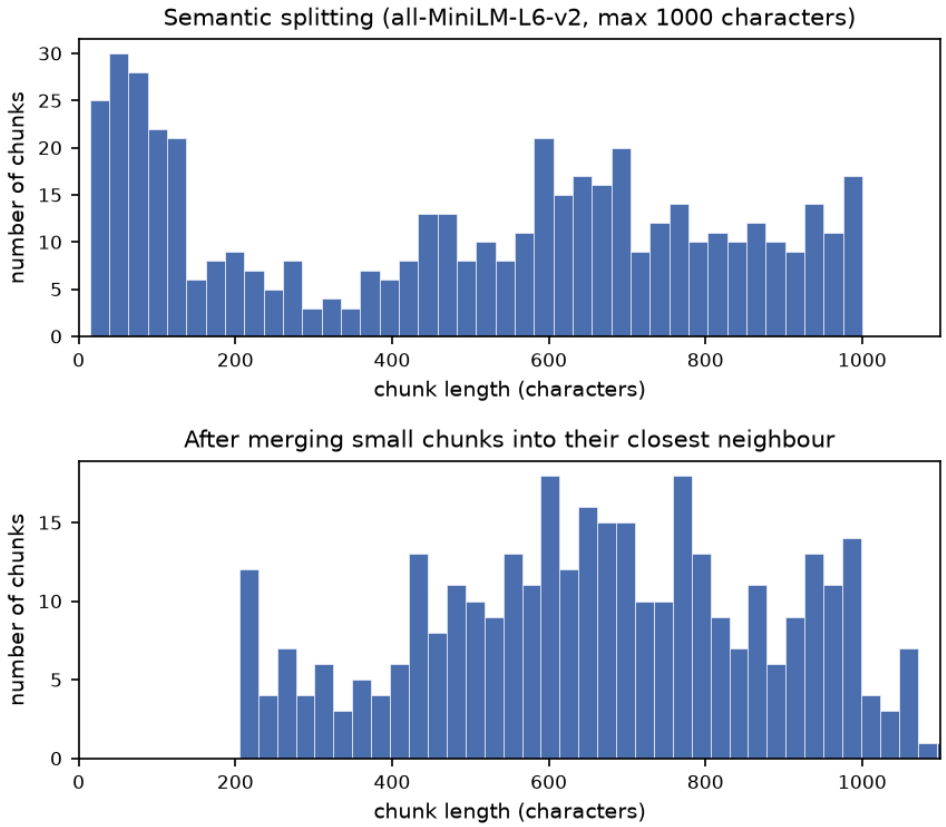


Figure 7.8: Semantic splitting of A Study in Scarlet *(all-MiniLM-L6-v2, maximum 1000 characters). Top: sizes spread across the whole range because boundaries follow topic shifts, leaving a residue of tiny unmerged chunks. Bottom: folding each small chunk into its closest neighbour clears the fragments and centres the distribution in the mid-range.*

Hierarchical Chunking

The methods above each pick one chunk size, and that single choice cannot serve both retrieval and generation well. Hierarchical chunking refuses the choice and stores the document at more than one level of granularity at once. This is the arrangement most suited to RAG.

The idea is to split coarsely, then finely, and keep both. Divide the document by large structural units such as headings or sections. Within each unit, apply a finer splitter: sentence-based, fixed-token, or semantic. Store both levels, and give each a job. Small chunks produce accurate embeddings and drive retrieval; the larger section they belong to is passed to the language model as context, so generation sees the full surrounding passage even though retrieval matched on a precise fragment.

```
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter,
    RecursiveCharacterTextSplitter,
)

header_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "H1"), ("##", "H2"), ("###", "H3")]
)
```

```
sections = header_splitter.split_text(long_markdown_text)

token_splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
final_chunks = []
for sec in sections:
    for idx, sub in enumerate(token_splitter.split_text(sec.page_content)):
        final_chunks.append({
            "content": sub,
            "metadata": {**sec.metadata, "sub_chunk": idx},
        })
```

Retrieval runs against the small `sub` chunks; the stored metadata lets the system recover the parent section and hand that larger unit to the generator.

Query-Aware Chunking

Every method so far splits the document before any query arrives, which is why it is called static chunking. Yet different questions need different amounts of context. “When was Albert Einstein born?” wants one short, fact-bearing chunk; “Summarize how vector search works” wants a whole section. Query-aware, or adaptive, chunking decides how much context to assemble only once it has seen the query.

The pattern builds on hierarchical storage. Preprocess the document into small, embedding-friendly chunks. When a query arrives, interpret its intent, retrieve the top chunks as usual, and then assemble context to match the intent: for a factual question, pass only the one or two highest-scoring chunks; for a summary, combine neighbouring chunks or return the whole parent section; for a multi-hop question, gather related chunks from several sections and stitch them together.

This reduces noise for simple questions and supplies enough context for complex ones, while sending the language model fewer tokens when fewer will do. Frameworks like LangChain, LlamaIndex, and Haystack already implement early versions, retrieving small chunks to score relevance and promoting larger parent sections when needed. As context windows keep growing, query-aware chunking shifts from a workaround for size limits toward a way to optimize relevance, cost, and reasoning quality.

Which Strategy Is Best?

After six methods, the natural question is which one to use, and the honest answer is that there is no universal winner. The best strategy depends on the documents, the questions users ask, and how much context the reader can absorb. What we can do is measure, but the metrics from Performance Evaluation do not transfer as cleanly as they first appear.

The difficulty is that relevance is defined for a query, while chunking happens before any query exists. The chunk boundaries, fixed in advance, then decide how well retrieval can ever do. If an answer straddles a boundary, no single chunk holds it whole, and recall suffers no matter how good the retriever is. If a chunk buries one relevant sentence in paragraphs of unrelated text, the document looks relevant but the context handed to the generator is dilute. Chunking quality is really a question of how well boundaries line up with answers, and a histogram of chunk sizes, useful as the plots above are for spotting tiny or oversized chunks, cannot tell us that.

The dependable way to compare strategies is therefore to measure their effect on the task, holding the rest of the pipeline fixed and varying only the chunking. Against a set of questions with known answers, the key metric is answer recall at k: the share of questions for which at least one of the top k retrieved chunks contains the answer. It counts a hit per question, unlike the document recall of Performance Evaluation, which counts how many of all relevant documents were found, and it is the form of recall that bad boundaries break. A second axis is context precision, or token efficiency: how much irrelevant text rides along in those chunks, since large chunks win recall but waste the context budget and can distract the reader. The final judge is the end-to-end answer quality, measured with the faithfulness and context metrics of Evaluating RAG.

A cheaper, label-free diagnostic is sometimes enough during development: compare the similarity of sentences within a chunk against the similarity across a boundary. A good split keeps sentences inside a chunk alike and drops sharply at the boundary, which is exactly the cohesion that semantic splitting optimizes. It is a useful smell test, but no substitute for measuring the task itself. In the end the right chunking is the one that scores best on our own questions and our own reader, not the one with the tidiest distribution.

Hands-on: Chunking strategies

Split one long document with fixed-size, sentence, structural, semantic, hierarchical, and query-aware chunking, compare the chunk-size distributions, and rank the strategies by answer recall at k against the context each one costs. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Result Generation

By the time the pipeline reaches this stage, retrieval and reranking have produced a short list of relevant passages. We could stop and show that list, but the point of RAG is to go further and return an answer. The stage that turns passages into an answer comes in three forms, and the difference between them is what the user gets back. We carry one running query through all three:

> Who won the Nobel Prize in Physics in 2025?

A **reader** marks the answer inside a passage and returns that span verbatim. A **generator** writes a new, direct answer in natural language from the retrieved passages. A **synthesizer** summarizes across several documents, gathering evidence over multiple rounds to build an answer no single passage contains. The three trade faithfulness for flexibility: the reader cannot say anything its source did not, which makes it safe but rigid; the synthesizer can assemble an answer from many sources, which makes it powerful but the hardest to keep honest.

Choosing Among the Three

The right form depends on the question and the stakes, not on which is most sophisticated. Four factors decide it.

The size and quality of the context comes first. A reader works best on small, highly relevant passages where the answer is present verbatim; a generator or synthesizer suits broad, diverse context that must be combined. Question complexity follows: a simple fact wants a reader, while an open-ended or multi-part question wants a generator or synthesizer. User intent matters too: a user who wants an exact quote or citation is served by a reader, one who wants an explanation by a generator. Finally, risk tolerance can override the rest. In legal or medical settings, where an invented detail is dangerous, a reader's inability to add anything is a feature, not a limitation.

In a production system the choice is made per query. A classifier or the agent itself reads the query, picks the form, and plans the rest of the pipeline accordingly, which is the routing we return to in Agentic RAG.

The Reader

A reader finds the span of text in a passage that answers the question and marks it. It generates nothing; it predicts where the answer starts and where it ends. This is the mechanism behind a highlighted search result, and the standard design, built on BERT, is shown in Figure 7.9.

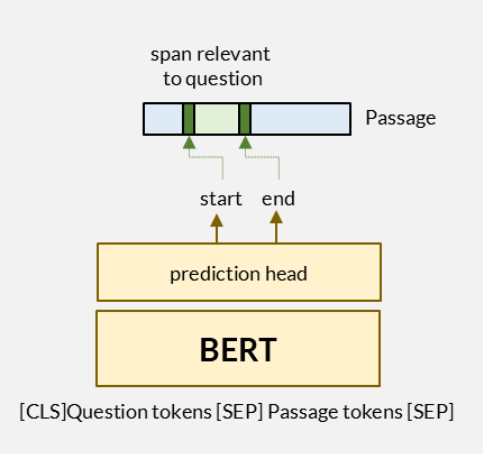


Figure 7.9: A reader as BERT-based extractive question answering. The concatenated question and passage are encoded, and a prediction head scores each passage token as a possible start or end of the answer span.

The question and passage are joined into one sequence, marked by special tokens: `[CLS]` begins the input and `[SEP]` separates question from passage. BERT encodes the whole sequence, so each token's representation reflects both the question and the rest of the passage. A lightweight prediction head then assigns

every passage token two scores, one for how likely it is to begin the answer and one for how likely it is to end it. The answer is the span whose start and end scores sum highest. The model is fine-tuned on datasets such as SQuAD, which provide the correct start and end positions for each question-passage pair, teaching the head to locate answer spans.

In code, we encode the question and passage together, run the model, and read off the span between the highest-scoring start and end tokens:

```
import torch
from transformers import AutoTokenizer, AutoModelForQuestionAnswering

name = "distilbert-base-uncased-distilled-squad"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForQuestionAnswering.from_pretrained(name)

question = "Who won the Nobel Prize in Physics in 2025?"
context = (
    "John Clarke, Michel Devoret, and John Martinis won the 2025 Nobel "
    "Prize in Physics for their discovery of macroscopic quantum tunnelling."
)

enc = tokenizer(question, context, return_offsets_mapping=True, return_tensors="pt")
offsets = enc.pop("offset_mapping")[0]
with torch.no_grad():
    output = model(**enc)

start = output.start_logits.argmax()   # highest-scoring start token
end = output.end_logits.argmax()       # highest-scoring end token
answer = context[offsets[start][0]: offsets[end][1]]
print(answer)
# John Clarke, Michel Devoret, and John Martinis
```

The model returns the exact span from the passage, with the start and end positions peaking sharply. Its strength is also its ceiling: it can only return text that is literally present, so it cannot combine two half-answers or rephrase an awkward one. This is why the Featured Snippet of The RAG Pipeline could surface a direct answer without any language model at all: a reader only has to point, not to write.

The Generator

A generator uses a language model to compose an answer token by token from the question and the retrieved passages. Because it writes rather than copies, it can merge facts from several passages into one fluent, direct response. Asked “what contributions led to the 2025 Nobel Prize in Physics?”, it can draw on several documents and produce a coherent paragraph that no single passage states outright.

That freedom is also where hallucination enters, so the prompt is where a generator is controlled. Effective prompts keep the model grounded in the retrieved text and inside the intended domain. A well-built prompt states the task clearly (answer, summarize, or explain); supplies the retrieved passages as the only source of truth; reminds the model to answer only from that context and not from prior knowledge; restricts the scope to the application’s domain; specifies the output format, such as a short answer, a paragraph, or JSON; optionally sets tone or audience; and tells the model what to do when the answer is not in the context, so it can say it does not know instead of inventing one.

How these pieces are arranged matters as much as whether they are present. During training a model learns particular ways of attending to the instruction, the question, and the context, and it is sensitive to their order and phrasing: burying the instruction after a long context, or leaving a key constraint implicit, can quietly degrade the answer even when all the information is there. Getting this right is the craft known as prompt engineering, and it is enough of a craft that we do not attempt to teach it here. The point to carry away is that the prompt is not boilerplate but a real lever on quality, worth iterating on. One practical safeguard is meta-prompting: using a language model to draft and refine the prompt itself against examples,

which keeps prompts effective as models and requirements change rather than relying on a hand-tuned string that silently goes stale.

To make the building blocks concrete, the skeleton below lays them out in the structure Claude follows most reliably.

A grounded-answer prompt skeleton (optional reading)

The retrieved passages go in tagged blocks and before the question, the grounding and citation rules are stated plainly, and the key constraint is repeated once after the data. The parts in braces are filled in at request time. This is deliberately shorter than a production prompt; it shows the blocks, not the polish.

```
PROMPT = """
You answer questions about {domain} using only the passages below.
Cite the passages you rely on by id, like [1]. If the answer is not in
them, say you don't know rather than guessing.

<passages>
{passages}
</passages>

<question>
{query}
</question>

Answer in {format}, and base every claim on the passages above.
"""
```

The parts map onto the checklist above: task and scope in the first line, the retrieved context in a tagged block, the question last, and a closing grounding reminder so the key instruction is the last thing the model reads.

A fluent answer is not a grounded one

A language model generates equally fluent text whether or not the facts are in its context, so fluency is no evidence of correctness. A generator must therefore be paired with checks that each claim it makes is supported by a retrieved passage, and the system should prefer saying "I don't know" to emitting an unsupported but plausible answer. Enforcing this across the whole system is the job of guardrails, covered in Agentic RAG.

The Synthesizer

A synthesizer keeps the generator's fluency and adds iteration. For questions that need evidence assembled across several steps, one retrieval pass is not enough, so the synthesizer loops: retrieve, read, decide what is still missing, form a follow-up query, and retrieve again. This is the multi-hop loop from The RAG Pipeline, and it is an early, hard-wired version of the agent loop in the next section.

A question such as "where do the 2025 Nobel laureates in physics work today?" shows the pattern. The answer lies in each laureate's own biography, but the question names no laureate, so a single retrieval pass returns what its words point at: prize lists and statistics, pages about the prize rather than about the people. The synthesizer's first hop retrieves passages about the prize and extracts the laureates' names. The second hop turns each name into a new query for that person's career. A generator then combines the hops, reconciles any contradictions, and summarizes them into one coherent answer. On the Nobel knowledge base of the chapter's demos, with the same budget of nine passages for both, a single pass reaches only one of the laureates' institutions, Santa Barbara, while the two hops reach Berkeley, Yale, Santa Barbara, Saclay, and Google. Hops pay off when the question does not name what the answer is about. The payoff

is completeness on hard questions; the cost is latency and complexity, since each hop is another round trip through retrieval and reading.

Citations

The ability to cite sources is what most clearly separates a RAG answer from a bare language-model answer. By linking each claim to the passage it came from, a citation-aware system becomes transparent and checkable, which is the foundation of user trust.

Citations are preserved through the pipeline rather than added at the end. During retrieval, each passage keeps its metadata: a passage identifier for internal tracking, the source document, its URL or location, the retrieval score, and the original text. As the answer is produced, the system records which passages contributed, creating an auditable link between answer and evidence. The final output pairs the answer with structured citation details:

```
{
  "answer": "John Clarke, Michel H. Devoret, and John M. Martinis",
  "confidence": 0.95,
  "citation": {
    "passage_id": "passage_001",
    "source": "Nobel Prize Official Website",
    "url": "https://nobelprize.org/prizes/physics/2025/",
    "original_text": "The 2025 Nobel Prize in Physics was awarded to John Clarke,
Michel H. Devoret, and John M. Martinis for their discovery of macroscopic quantum
mechanical tunnelling and energy quantization in an electric circuit.",
    "retrieved_date": "2025-10-07"
  }
}
```

This structure pays off three ways. It lets users verify an answer by following the citation to the original material. It builds trust, because answers are visibly grounded in evidence rather than asserted. And it gives developers a trace to debug and refine the retrieval and reading stages, so the system improves over time rather than failing silently.

Hands-on: Reader, generator, and synthesizer

Run all three forms on the same retrieved passages: a BERT reader that marks the answer span, a generator with a grounded prompt, and a two-hop synthesizer against a single pass with the same budget, then check every citation quote against its passage. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Agentic RAG

Retrieval-augmented generation has turned out to be one of the most useful patterns in applied AI, and enterprises have adopted it quickly. It combines the recall of a retrieval system with the fluency of a generator, keeps answers grounded in sources the organization controls, and needs no training or fine-tuning to absorb new or private information. A document added to the index this morning can be cited in an answer this afternoon.

What it has not removed is the pipeline. Everything so far is wired by hand into a fixed sequence, and that sequence encodes a single intent. A question-answering flow, a summarization flow, and a comparison flow are three different pipelines, each stitched together in code. A framework such as the Haystack pipeline of the first section takes away much of the wiring, but the shape of the problem remains: one pipeline per intent, and every change is a code change that must be deployed. Worse, anything ad hoc falls outside the design. Answering "who won the 2025 Nobel Prize in Physics, and where does each laureate work?" is not a single retrieval; it is one lookup for the winners, then a further lookup per winner for their affiliation. A fixed pipeline cannot express that chain unless someone anticipated it and coded it, and the alternative, making the user ask each follow-up by hand, defeats the point of a direct answer.

The Agent Loop

Agentic RAG drops the fixed sequence and gives control to the model. Instead of executing stages in a predetermined order, an agent runs a loop: it reasons about what the request needs, acts by calling a tool, observes the result, and decides whether it is done or must go round again. Figure 7.10 shows the shape.

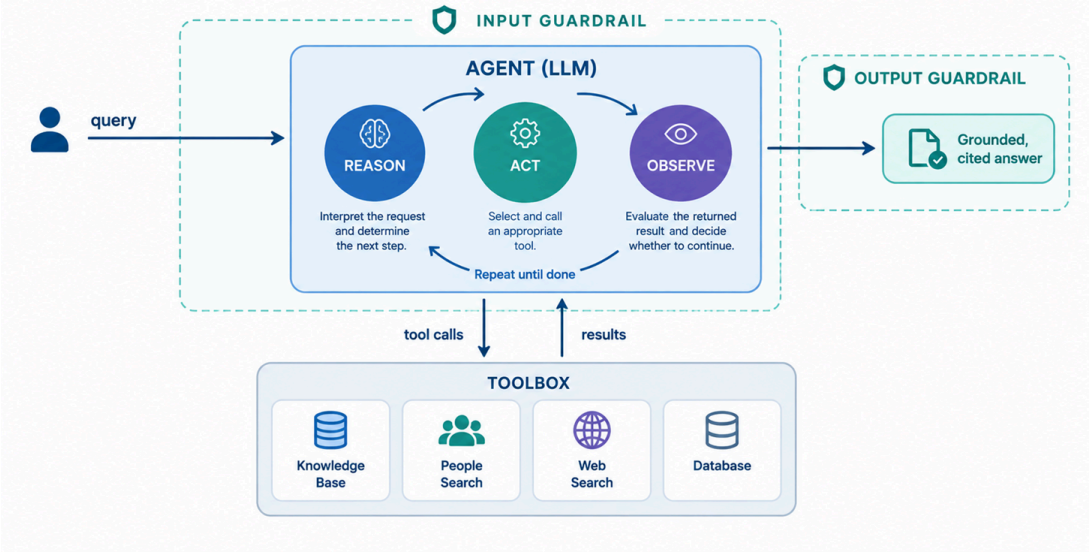


Figure 7.10: An agentic RAG system: the agent runs a reason-act-observe loop, calling tools from a toolbox until it can answer, bounded by input and output guardrails.

The three-step cycle, reason then act then observe, is the ReAct pattern, and the concrete mechanism behind the act step is tool calling, also known as function calling: the model emits a structured request naming a tool and its arguments, the system runs the tool, and the result returns as the next observation. The agent chooses each action as it goes, so the number of steps and the tools used are decided by the query rather than fixed in advance. A stop condition ends the loop, either when the agent judges it has enough to answer or when an iteration budget is reached, the latter a guard against a loop that never terminates.

Because the control flow is decided as it runs, the same loop serves both a known, repeatable flow and a one-off ad-hoc request. It also lets the agent recover from trouble that would halt a fixed pipeline. If the knowledge base is unavailable, it can try another source; if the evidence gathered so far is incomplete, it can fall back to a web search rather than return a half-answer.

Walking the Loop: the 2025 Physics Nobel

To see the loop earn its keep, take the multi-part question from above and give the agent two tools: a Wikipedia knowledge base that returns article text, and a people-search tool that returns structured JSON about a person. A third tool, web search, is available as a fallback.

> Who won the 2025 Nobel Prize in Physics, and which universities are they affiliated with?

Example: the agent loop on the Nobel question

1. **Intent detection (reason).** The agent reads the question and recognizes two things: it asks for current facts it cannot answer from memory, and it has two parts, the winners and their affiliations. No separate classifier is needed; the intent is inferred inside the loop. Had the question been vague, such as "tell me about the physics Nobel", the agent could ask a clarifying question or infer the intent from the conversation so far, using meaning rather than keyword rules.
2. **Query transformation (act).** It calls the Wikipedia tool with a well-formed query, `"2025 Nobel Prize in Physics"`, rather than passing the user's raw sentence. The result it observes is an article naming John Clarke, Michel Devoret, and John Martinis.
3. **Search expansion (reason, then act again).** The affiliations are still missing, so the agent loops. For each laureate it calls people-search, typically with one call per name in a single step, and the directory returns structured data such as `{"name": "John Clarke", "affiliation": "University of California, Berkeley"}`. This per-winner chain is exactly the multi-hop flow a fixed pipeline could not express. The directory, like most real ones, is incomplete: it has no record for John Martinis, so for him the agent tries another way, a different spelling, the knowledge base, or, as a last resort, web search.
4. **Ad-hoc flow.** Nobody coded a winners-then-affiliations pipeline; the agent assembled it from the question. A flow that recurs often could instead be handed to the agent as a fixed skill, but it does not have to be.
5. **Final answer.** When the agent observes that the evidence is complete, it stops calling tools and generates a grounded, cited answer naming the three laureates and their universities, with the source of each fact.

The shift is from a Haystack-style pipeline hardcoded for one intent to a dynamic flow the agent assembles from a description of the tools it may use. We stop writing the sequence and start declaring the capabilities, and intent detection, once a stage of its own (see Advanced Text Processing), becomes simply the loop's first thought.

Implementing the Loop

The loop itself is short. What makes it work is the tool declaration: each tool has a name, a description, and its parameters, and the description is what the model reads when it decides which tool to call. Declaring the tools is how we tell the agent what it may do.

```
def schema(arg):
    """JSON Schema for a tool with one required string argument."""
    return {"type": "object", "properties": {arg: {"type": "string"}}, "required":
[arg]}

TOOLS = [
    {"name": "wikipedia_search",
     "description": "Search a Wikipedia knowledge base about the Nobel Prizes and
return "
                    "the text of the 3 best matching passages with their source.",
     "input_schema": schema("query")},
    {"name": "people_search",
     "description": "Look up a person in the people directory. Returns JSON with
name, "
```

```
                    "affiliation, and position, or an error if there is no record.",
     "input_schema": schema("name")},
    {"name": "web_search",
     "description": "Search the web. Returns titles and short snippets. "
                    "Use only when the other tools return nothing.",
     "input_schema": schema("query")},
]

SYSTEM = ("Answer with the tools provided. Base every statement on tool results "
          "and name the source of each fact. If the tools do not give the answer, say
so.")

def run_agent(question, max_steps=8):
    messages = [{"role": "user", "content": question}]
    for _ in range(max_steps):                                  # iteration budget
        reply = llm(messages, tools=TOOLS, system=SYSTEM)       # reason
        messages.append(reply.message)
        if not reply.tool_calls:                                # no further action:
done
            return reply.text
        results = [tool_result(call.id, call_tool(call.name, call.arguments))
                   for call in reply.tool_calls]                # act
        messages.append({"role": "user", "content": results})  # observe
    return "I could not complete the answer within the step budget."
```

Each tool is declared with a JSON Schema of its arguments, the format the model provider expects; the system prompt sets the grounding rule for the whole run. The function `llm` sends the conversation and the declarations to the model, here Claude, and returns its reply with any tool calls it requests; `call_tool` runs the named tool; and `tool_result` wraps the output as an observation that refers back to the call it answers. Each pass through the loop is one reason-act-observe cycle: the model either requests tool calls or replies without one, which ends the loop. One reason step can request several calls at once, such as one people-search per laureate, so all their results return together in a single message. The growing message list is the agent's working memory, so in the Nobel example the laureates' names from the first tool call are in view when the agent decides to look up their affiliations. The `max_steps` budget is the stop condition that keeps a confused agent from looping forever.

A recurring flow can be handed to the same loop as a skill, a short description of the steps to follow, placed in the system prompt or loaded when the intent calls for it; the loop is unchanged. Frameworks package this pattern, including Haystack's agent component and graph-based libraries such as LangGraph, adding conveniences such as tool schemas generated from Python functions and tracing of each step. The guardrails below hook into the same two places in the loop: before `call_tool` runs, and before the final `return`.

Guardrails

Handing control to a model raises the stakes, because the model now decides what to search, which tool to call, and what to answer. Guardrails are the checks that keep those decisions safe and grounded, and they sit at the two edges of the loop, as the dashed boxes in Figure 7.10 show.

Input guardrails act before the agent takes any action. They reject queries that are unsafe or outside the system's scope, so the agent neither wastes a tool call nor answers a question it has no business answering. A support assistant scoped to one insurance product, asked about a competitor's policy, should decline cleanly rather than improvise.

Output guardrails act before the answer reaches the user. They check two things: that the answer still serves the original intent and has not drifted over the course of the loop, and that every claim is supported by a retrieved result. An answer that goes beyond the evidence is a likely hallucination and must be caught; when a check fails, the safe response is to fall back to uncertainty rather than emit a confident but unsupported answer.

Guardrails run at every hop, not once

In a fixed pipeline it is tempting to check the output only at the end. In an agent loop the model acts repeatedly, so an unsafe tool call or a drifting intermediate result can derail the steps that follow. Apply the input check before each action and the grounding check on each intermediate result, not only on the final answer.

The check before each action needs no model; it is plain code. Before `call_tool` runs, the loop verifies that the request names a declared tool, passes exactly the declared arguments, and leaves none of them empty. A blocked call is not silently dropped: the reason it was blocked becomes the observation, so the model sees the refusal and can choose another action.

Key Findings

Step back and the significance is hard to miss. A single generic loop, given the right data and tools, can answer the whole range of queries that previously demanded a separate pipeline each. The agent supplies the control flow; the developer supplies the tools and the data behind them.

This is the engine behind the current wave of agentic assistants. Personal AI systems such as OpenClaw and Hermes push the idea to its limit, running one loop for everything a user might ask, in the same spirit as Claude Code. The retrieval components of this book, chunking, embeddings, hybrid retrieval, reranking, and reading, are all still present, but they have become tools the agent may call rather than stages a programmer must wire.

The deeper change is in who builds these systems and how. For the first time, assembling a retrieval system is less a programming task than a matter of defining the process and integrating the tools, work that increasingly happens in low-code or no-code environments. Describing what the agent may do, and trusting the loop to decide when to do it, has proven a more flexible way to build than wiring the stages by hand.

Hands-on: Agentic RAG

Run the reason-act-observe loop with Claude and three tools on the Nobel question, read its trajectory step by step, see the same loop answer questions of different shapes, cut it short with the step budget, and add scope, tool-call, and grounding guardrails (recorded runs replay without AWS access). Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Evaluating RAG

Every section of this chapter has added choices. We can chunk by size, by sentence, by structure, or by meaning; embed with one model or another; weight BM25 against dense retrieval; rerank or not; retrieve five passages or twenty; pick a generator and a prompt; and let an agent loop for two steps or ten. Classical retrieval asked how well one ranking function performs on a benchmark. A RAG system forces a broader question: of all these combinations, which one serves the users best, and how would we know?

Precision and recall are part of the answer but no longer all of it. Three further dimensions enter: the quality of the generated answer, the time it takes to produce, and what it costs. The table below shows how the main groups of choices map onto what they mainly affect.

Choice	Example options	Mainly affects
Document processing	chunking strategy, chunk size, overlap	retrieval recall, index size
Retrieval	embedding model, BM25 weight, top-k	recall, precision, retrieval latency
Reranking	reranker model, candidates reranked	precision, latency
Generation	generator model, prompt, context size	answer quality, token cost
Agent control	tools available, iteration limit	answer quality, number of tool calls, total latency

Retrieval Still Sets the Ceiling

A generated answer can never be better than the evidence it is built from, so the retrieval metrics of Performance Evaluation still apply, recast to describe the retrieved context rather than a result list. Context recall asks whether everything needed to answer the question was retrieved. If a required fact never reaches the generator, no model and no prompt can recover it; the best it can do is admit the gap, and the worst is to invent the missing piece. Context precision asks how much of the retrieved context is actually relevant. Irrelevant passages are not harmless padding: they compete for the generator's attention, and closely related but wrong evidence is the most dangerous kind.

Example: context recall and precision on the Nobel question

Recall the running question: "Who won the 2025 Nobel Prize in Physics, and which universities are they affiliated with?" A complete answer needs four pieces of evidence: one passage naming the three laureates and one affiliation for each of them.

A fixed pipeline that retrieves the top five chunks for the whole question returns the passage naming the winners, one laureate's affiliation, two chunks about the 2025 Nobel Prize in Chemistry, and one about the award ceremony. Two of the four required pieces arrived, so context recall is $2/4 = 0.5$; two of the five chunks are relevant, so context precision is $2/5 = 0.4$. The answer will be missing two affiliations, and the Chemistry chunks invite the generator to attach a chemist's university to a physicist.

The agent from Agentic RAG first retrieves the winners and then looks up each laureate in turn. It gathers all four pieces and nothing else: context recall and context precision are both 1.

Recall needs to know the full set of evidence an answer requires, so it depends on a labelled reference. Precision over the retrieved context can often be judged without one, by asking whether each passage bears on the question. Both trade off as they do in classical retrieval: retrieving more raises the chance of capturing every required passage but dilutes the context with noise.

The Price of an Answer

A ranked list arrives in milliseconds and costs a fraction of a cent. A generated answer can take seconds, and every request consumes tokens that someone pays for. An agent multiplies both, since each pass through the loop is another model call and possibly another tool call. The bar is therefore higher than for classical

search: a RAG answer must be clearly better for the user, not merely different, to justify the extra latency and cost.

Latency should be measured as a high percentile rather than an average. The 95th percentile, or p95, is the time within which 95 per cent of requests complete, and it reflects the slow cases users actually notice. Cost is best expressed per answered question, combining input tokens, output tokens, and tool charges.

For an agent, the path to the answer matters as well as the answer itself. A correct answer reached in two tool calls is better than the same answer reached in ten, even though both are equally correct. Evaluating the trajectory means recording how many iterations the loop took, which tools it called, whether each call was the right one, and whether it kept searching after it already had enough evidence.

Measuring Answer Quality

The hardest dimension to measure is the quality of the answer itself, because it is free text rather than a ranked list. It is useful to separate four questions, which overlap but are not interchangeable.

- **Correctness**: is the answer factually right? This is judged against a trusted reference answer or a list of expected facts.
- **Faithfulness**: is every claim in the answer supported by the retrieved context? This measures grounding.
- **Relevance**: does the answer address the question that was asked, rather than a neighbouring one?
- **Completeness**: does the answer contain all the facts the question requires?

Faithfulness is the dimension specific to RAG, since it ties the answer to the evidence the system actually retrieved.

Key Formula: Faithfulness

$$\text{faithfulness} = \frac{|\ \{\text{claims in the answer supported by the context}\}\ |}{|\ \{\text{claims in the answer}\}\ |} \tag{7.1}$$

The fraction of the answer's claims that the retrieved passages support. A value of 1 means every claim is grounded; a lower value means the generator added content the evidence does not back.

Example

The answer reads: "The 2025 Nobel Prize in Physics went to Clarke, Devoret, and Martinis, who shared 11 million Swedish kronor." The retrieved context names the three laureates but says nothing about the prize money. Of the two claims, one is supported, so faithfulness is $1/2 = 0.5$. The prize amount happens to be correct, yet it still lowers faithfulness: the generator took it from its own memory rather than from the evidence, and a claim taken from memory can just as easily be wrong.

Faithful is not the same as correct

Faithfulness measures agreement with the retrieved context, not with the world. A true claim that the context does not support lowers faithfulness, as in the example above. And a perfectly faithful answer can be wrong if the retrieved documents are outdated or mistaken. Measure correctness separately, against a reference.

The four dimensions of answer quality, correctness, faithfulness, relevance, and completeness, are measured in two modes. Reference-based evaluation compares the answer with a trusted expected answer; it gives the strongest signal of correctness and completeness but requires someone to write the references. Reference-free evaluation judges faithfulness and relevance from the question, the answer, and the retrieved context alone, which scales to thousands of production requests where no reference exists. A sound evaluation uses both: a smaller reference set for correctness and a larger reference-free sweep for grounding.

Most of these judgements are made by another language model acting as an evaluator, an approach called LLM-as-a-judge. For faithfulness, the judge splits the answer into individual claims and checks each against

the context. RAGAS packages the common metrics in this way: context precision and recall, faithfulness, and answer relevance, along with reference-based correctness.

Validate the judge before trusting it

An LLM judge is itself a model, with its own blind spots: it can miss subtle factual errors, favour longer answers, and change its verdicts when the evaluation prompt changes. Check its scores against a sample of human judgements before relying on them, and keep the judge model and prompt fixed while comparing systems. Where a rule can decide, such as a date, an amount, or whether a cited passage exists, use a deterministic check instead of a judge.

Measuring retrieval and answer quality separately also tells us where to look when something goes wrong.

Retrieval quality	Answer quality	Likely diagnosis
Low	Low	Fix retrieval: chunking, indexing, or query formulation.
High	Low	Fix generation: the prompt, the model, or how the context is used.
Low	High	Suspicious: the model may be answering from memory rather than evidence.
High	High	Working; look for savings in latency and cost.

These are heuristics rather than proofs, but they turn a single disappointing score into a direction to investigate.

Comparing Two Configurations

The practical question is usually narrower than "which configuration is best". We have a system A and a proposed change B, perhaps a new chunking strategy, a reranker, or an agent loop, and we want to know whether B is better. The method is a controlled experiment.

Start from a fixed benchmark: a set of representative questions, each with a reference answer and the evidence it requires, built the same way as the test collections of Designing a Retrieval Benchmark. Run A and B on exactly the same questions, changing only the one component under test, and record for every question the retrieved context, the answer, the latency, the tokens used, and for an agent the tool calls. Then compute every dimension for both systems. The sketch below shows the shape of such a harness; the `judge_*` functions stand for evaluators such as those RAGAS provides.

```
def evaluate(system, benchmark):
    rows = []
    for item in benchmark:   # question, reference answer, required evidence
        start = time.perf_counter()
        run = system.answer(item.question)   # answer, retrieved, tool_calls, tokens
        latency = time.perf_counter() - start   # stop the clock before evaluating
        rows.append({
            "context_recall": recall(run.retrieved, item.evidence),
            "context_precision": precision(run.retrieved, item.evidence),
            "correct": judge_correct(run.answer, item.reference),
            "faithfulness": judge_faithful(run.answer, run.retrieved),
            "latency_s": latency,
            "tokens": run.tokens,
            "tool_calls": len(run.tool_calls),
        })
    return rows

results_a = evaluate(system_a, benchmark)
results_b = evaluate(system_b, benchmark)
```

A higher average for B is not yet proof that B is better. Benchmarks are small, and some questions are easy or hard for reasons unrelated to the change. Because both systems answered the same questions, the

right comparison is paired: look at the questions where the two systems disagree, as in the paired tests of Evaluating Ranked Results.

Example: is B really better?

On a benchmark of 50 questions, system A answers 38 correctly (76 per cent) and system B answers 44 (88 per cent). Is B better, or did it get lucky? The test proceeds in four steps.

1. **Keep only the disagreements.** Both systems get 36 of the same questions right and 4 of the same questions wrong. These 40 questions say nothing about which system is better, so we set them aside. What remains are the 10 questions where the systems disagree: B is right and A wrong on 8, A is right and B wrong on 2.
2. **Assume there is no difference.** If B were no better than A, each disagreement would be equally likely to go either way, like the toss of a fair coin. The number of disagreements won by B would then follow a binomial distribution: the probability that B wins exactly k of the 10 is $\texttt{binom}\ 10k/2^{10}$, where $\texttt{binom}\ 10k$ counts the ways to choose which k of the 10 questions B wins.
3. **Measure how extreme the observed split is.** The probability of exactly 8 to 2 is $45/1024 \approx 0.044$, but that number is misleading on its own. Every individual outcome is unlikely, even the most typical one: an even 5 to 5 split has probability $252/1024 \approx 0.25$, and the probability of any single exact outcome shrinks further as the number of disagreements grows. The meaningful question is how often chance alone produces a result at least as one-sided as the one we saw, so we add up 8 to 2, 9 to 1, and 10 to 0: $(45 + 10 + 1)/1024 \approx 0.055$.
4. **Count both directions.** Before the experiment we did not know which system would come out ahead, and a 2 to 8 split in favour of A would have been just as surprising. Since the distribution is symmetric, we double the probability:

$$p = 2 \cdot \frac{\texttt{binom}\ 108 + \texttt{binom}\ 109 + \texttt{binom}\ 1010}{2^{10}} = 2 \cdot \frac{45 + 10 + 1}{1024} \approx 0.11. \tag{7.2}$$

If A and B were truly equally good, a split at least this uneven would still occur about one time in nine. That is not rare enough to rule out chance, against the common threshold of 0.05, so the twelve-point gap is not conclusive. With the same pattern on a benchmark of 100 questions, B winning 16 of 20 disagreements, the same calculation gives $p \approx 0.012$, and the improvement becomes convincing. The ratio is unchanged; what changed is the amount of evidence behind it.

Quality is only half of the decision. If B is more accurate but doubles the p95 latency or the cost per answer, whether it is better depends on what the application can afford, and that is a product decision the measurements inform rather than settle. A good habit is to fix the acceptable latency and cost before running the experiment, so the result cannot quietly move the goalposts.

Tooling helps with the bookkeeping. Metric libraries such as RAGAS and DeepEval supply the judges and turn quality thresholds into regression tests, and tracing tools such as Phoenix and LangSmith record each run so that a surprising score can be traced to the step that caused it. Running such comparisons systematically over many parameters at once, known as hyperparameter search, builds on exactly this A-versus-B foundation and goes beyond the scope of this book.

When Not to Use RAG

The cost argument above cuts both ways. RAG is not the default answer to every question-answering problem, and reaching for it when a simpler tool fits adds cost and latency for no benefit. Four situations call for a different approach.

- **The knowledge is stable and general.** The model already holds it. Asking a current model for a textbook definition or a historical fact gains nothing from retrieval and only adds a retrieval step that can fail.
- **The corpus fits in the context window.** When the entire relevant collection is small enough, long-context prompting can be simpler and more reliable than retrieval, because there is no chance of retrieving the wrong passage. As context windows grow, the size threshold for this keeps rising.

- **The task is to change behaviour, not to supply facts.** When the goal is a different style, tone, or output format, fine-tuning is the right tool, since RAG changes what the model reads, not how it writes.
- **The question maps onto structured data.** For a query such as "how many orders shipped last week", a direct database or API query, possibly generated by a text-to-SQL model, gives an exact answer where retrieval over unstructured text would only approximate one.

An agent softens the choice, because it can decide per query: it can answer a stable fact from memory, send a structured question to a database tool, and reserve document retrieval for the questions that need it. The question then shifts from "RAG or not" to "which tool for this query".

Even where RAG is appropriate, it carries ongoing costs that the alternatives do not: an index to build and keep fresh, retrieval latency on every query, and a quality ceiling set by the corpus. If the document collection is noisy or out of date, retrieval faithfully supplies bad evidence, and the grounded answer is confidently wrong. RAG is the right choice when answers must draw on private, changing, or very specific knowledge and must be verifiable against sources, and the wrong choice when a cheaper tool already meets those needs.

Hands-on: Evaluating RAG

Compare a fixed pipeline with an agent on a small Nobel benchmark: context recall and precision, p95 latency and tokens per answer, correctness and faithfulness with an LLM judge checked against a rule, and a paired sign test on the questions where the two disagree. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Summary

How the Three Forms Compare

The result-generation stage is the clearest fork in a RAG design, so it is worth seeing its three forms side by side.

Form	How it answers	Best for	Main risk
Reader	Marks and returns the answer span verbatim from one passage	Exact facts, short passages, high-stakes answers	Cannot combine or rephrase; fails if no single span holds the answer
Generator	Writes a new, direct answer from several passages with a language model	Open-ended questions, fluent answers across sources	Hallucination; must be grounded by prompt and guardrails
Synthesizer	Loops retrieval and reasoning over several documents, then summarizes	Deep research needing evidence gathered across sources	Latency and complexity; drift across hops

The progression is one of flexibility traded against faithfulness. The reader cannot say anything its source did not, which makes it safe but rigid; the synthesizer can assemble an answer no single source contains, which makes it powerful but the hardest to keep honest. The form is chosen per query by the context available, the question's complexity, the user's intent, and how costly a wrong answer would be. In an agentic system that choice moves into the loop itself.

Key Takeaways

1. A ranked list leaves the user to find the answer, and a language model alone cannot answer what it never saw. RAG combines the two: retrieval supplies the evidence, the generator turns it into a direct answer, and grounding ties the answer to sources a user can check.
2. A RAG system reuses the retrieval machinery of earlier chapters, BM25, embeddings, vector search, and cross-encoder reranking, and adds two parts that exist because a model, not a human, now consumes the results: two-level chunking and a result-generation stage.
3. Chunking balances sharp embeddings against enough context for the generator. Two-level chunking keeps both, small chunks for matching and larger parents for generation, and no strategy wins everywhere: the right one is the one that retrieves the answers to our own questions.
4. The result-generation stage comes in three forms, a reader that marks a span, a generator that writes a direct answer, and a synthesizer that summarizes across documents. For the generator, the prompt is a real lever on quality, not boilerplate.
5. Agentic RAG replaces one hand-wired pipeline per intent with a single reason-act-observe loop over declared tools. The loop detects the intent, rewrites the query for each tool, follows multi-hop chains no one coded in advance, and stops within a step budget, with guardrails at its input and output.
6. Evaluating RAG means more than precision and recall. Retrieval still sets the ceiling, but answer quality has four dimensions, correctness, faithfulness, relevance, and completeness, and latency and cost must justify the answer. Two configurations are compared on the same benchmark with a paired test over the questions where they disagree.

Key Formulas

Key Formula: Faithfulness

$$\text{faithfulness} = \frac{|\ \{\text{claims in the answer supported by the context}\}\ |}{|\ \{\text{claims in the answer}\}\ |} \tag{7.3}$$

The share of the answer's claims that the retrieved passages support. It measures grounding, not truth: a correct claim the context does not back still lowers it, and a fully faithful answer can be wrong if the sources are.

Exam focus

- Why a ranked list falls short for single-answer questions, why parametric knowledge fails on recent or private facts, and when RAG is not the right tool.
- Two-level chunking, and why hybrid retrieval over two chunk levels must reconcile both rankings to a common unit before rank fusion.
- Reader versus generator versus synthesizer: mechanism, strengths, and failure modes.
- Fixed pipeline versus agent loop: what the loop takes over (intent, query transformation, multi-hop expansion, stopping) and where input and output guardrails apply.
- Correctness versus faithfulness versus relevance, and why an LLM judge must be validated against human judgements.
- Comparing two configurations: why only the disagreements count, and why the test asks for a result "at least as extreme" rather than the exact split.

Self-Check Questions

1. (Understand) A user asks a plain language model about an event last week and gets a confident but wrong answer. Explain which limit of parametric knowledge this illustrates and how RAG would address it.
2. (Understand) Dense retrieval runs on small chunks and sparse retrieval on their larger parents. Explain why their result lists cannot be fused with reciprocal rank fusion directly, and what has to happen first.
3. (Analyze) A RAG system splits documents into 2000-token chunks and retrieval precision is high, but generated answers are vague and incomplete. Diagnose the likely cause and propose a chunking change.
4. (Analyze) An answer scores 0.9 on answer relevance but 0.4 on faithfulness. What does this combination tell you about the system, and where would you look first?
5. (Apply) On a benchmark of 50 questions, a new configuration B answers 41 correctly and the baseline A answers 38. They disagree on 5 questions, of which B wins 4. Compute the two-sided sign-test probability and decide whether B is convincingly better.
6. (Evaluate) You are asked to build a question-answering feature over a product catalogue whose prices and stock change hourly. Argue whether RAG, fine-tuning, or direct structured-data queries is the better fit, and justify the trade-offs.
7. (Evaluate) A support assistant scoped to one insurance product starts answering questions about competitors' policies and sometimes states facts not found in its retrieved passages. Explain which guardrails are missing and where in the agent loop each one belongs.

Test Your Knowledge

Take the Chapter 7 Quiz ->>

Further Reading

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). **Retrieval-augmented generation for knowledge-intensive NLP tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.11401. The paper that named RAG and introduced the retrieve-then-generate architecture this chapter builds on.
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., et al. (2020). **Dense passage retrieval for open-domain question answering**. *EMNLP 2020*. arXiv:2004.04906. DPR, the dual-encoder dense retriever behind the dense-retrieval arm of the pipeline.
- Reimers, N., & Gurevych, I. (2019). **Sentence-BERT: Sentence embeddings using Siamese BERT-networks**. *EMNLP 2019*. arXiv:1908.10084. The sentence-embedding method used to embed chunks and queries, and to measure similarity in semantic chunking.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of deep bidirectional transformers for language understanding**. *NAACL-HLT 2019*. arXiv:1810.04805. The encoder behind both the reader and the cross-encoder reranker.

- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). **SQuAD: 100,000+ questions for machine comprehension of text**. *EMNLP 2016*. Dataset. The benchmark used to train and evaluate the reader of this chapter.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). **Lost in the middle: How language models use long contexts**. *Transactions of the Association for Computational Linguistics*. arXiv:2307.03172. Shows that models use information at the start and end of a long context far better than in the middle, one reason the order of a prompt matters.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). **ReAct: Synergizing reasoning and acting in language models**. *ICLR 2023*. arXiv:2210.03629. The reason-act-observe loop underlying the agentic RAG of this chapter.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). **RAGAS: Automated evaluation of retrieval augmented generation**. arXiv:2309.15217. The evaluation framework behind the faithfulness, answer relevance, and context precision and recall metrics of this chapter.
- deepset. **Haystack documentation**. docs.haystack.deepset.ai. Reference for the components and pipelines used to build the RAG pipeline in code.
- Firecrawl (2026). **OpenClaw vs Hermes Agent: Which one should you actually run?** *Firecrawl blog*. firecrawl.dev. A practical comparison of two personal assistants that run a single agent loop for everything a user asks.